

Sub-module 6

AXI4-LITE UART INTEGRATION

AXI4-Lite UART Integration

- ▶ How UART becomes a memory-mapped AXI4-Lite slave
- ▶ Register layout, write/read transaction flows, and firmware
- ▶ End-to-end: C code → AXI bus → serial bits on the wire
- ▶ For learners with UART and AXI4-Lite fundamentals

PREREQUISITES:

- ✓ UART protocol
- ✓ AXI4-Lite channels
- ✓ Basic C / MMIO

What We'll Cover

01

Why UART needs AXI slave interface — MMIO access pattern

02

uart_axi.v: 3 registers at 0x10000000 / 0x10000004 / 0x10000008

03

Write path: AXI → uart_axi → buf_valid → uart_tx → serial bits

04

Read path: uart_rx → rx_buf_valid → AXI read returns byte

05

Backpressure: AWREADY/WREADY = 0 when buf_valid = 1 (no data loss)

06

Firmware: fw/main.c uart_putc / uart_getc via volatile macros

07

Lab: make axi_uart → AXI master + uart_axi DUT → verify byte transfer

WHY AXI FOR UART?

The Mailbox Analogy

- CPU = Person wanting to send a letter
- UART registers = Mailbox slots
- AXI4-Lite = Standard postal interface
- AXI master/slave = Postman + Mailbox

CPU cannot directly drive serial pins — it uses memory-mapped I/O (MMIO). AXI4-Lite provides the standard bus interface for this.

AXI4-Lite Channels

AW

Address Write — CPU sends target address

W

Write Data — CPU sends byte to write

B

Write Response — Slave confirms completion

AR

Address Read — CPU requests register read

R

Read Data — Slave returns value

Register Map & Internal Architecture — uart_axi.v

Address	Offset	Name	Description
0x1000_0000	0x00	UART TX	Write wdata[7:0] → triggers TX serialisation
0x1000_0004	0x04	UART RX	Read returns {rx_buf_valid, rx_data[7:0]}
0x1000_0008	0x08	STATUS	Read returns {30'b0, rx_buf_valid, 1'b0}

Internal Architecture — uart_axi.v

AXI Write FSM → [1-byte TX buffer] → uart_tx (u_tx) ▶ tx_out

aw_got / w_got buf_data / buf_valid

uart_rx (u_rx, 2-FF sync) → rx_buf_data / rx_buf_valid → AXI Read FSM

⚠ Backpressure (Critical)

s_axi_awready = !aw_got && !buf_valid
s_axi_wready = !w_got && !buf_valid

→ CPU bridge STALLS when buffer is full → zero data loss

AXI Backpressure — How uart_axi Prevents Byte Loss

✗ Problem Without Backpressure

- CPU sends bytes faster than UART serialises
- Bridge time: ~534 cycles
- UART byte time: ~4340 cycles (115200 baud)
- Without BP: 2nd byte overwrites 1st → data corruption

✓ Solution in uart_axi.v

- When `buf_valid = 1`:
`s_axi_awready ← 0` (block new address)
`s_axi_wready ← 0` (block new data)
CPU bridge stalls → PicoRV32 waits automatically

AXI Handshake Timeline

`buf_valid=1`

`awready=0`
`wready=0`

`uart_tx.ready=1`

`buf_valid←0`

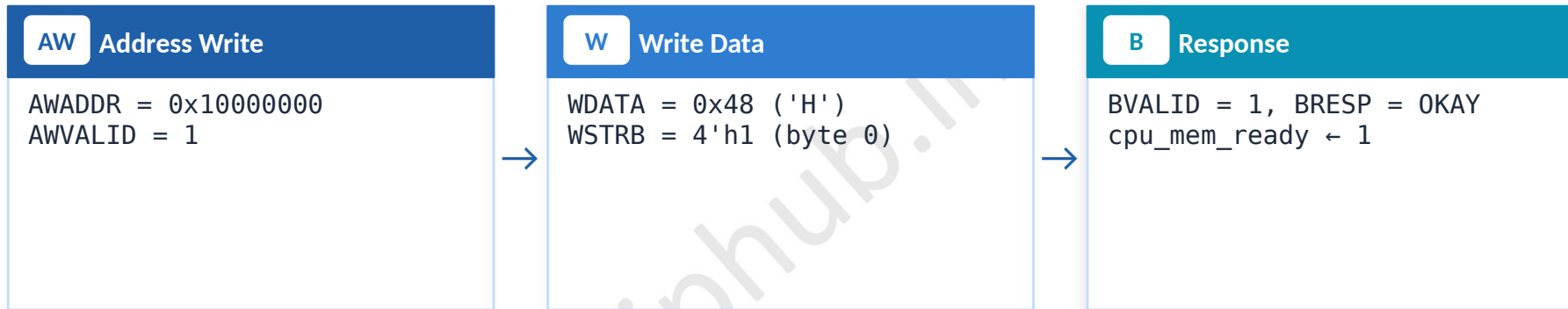
`buf_valid=0`

`awready=1`
`wready=1`

Next byte
Accepted

Handshake OK

AXI Write Transaction — `uart_putc('H')`



uart_axi.v Internal Response:

aw_got=1 → w_got=1 → buf_data=0x48, buf_valid=1 → bvalid=1 (OKAY) → uart_tx.valid=1

⚠ Backpressure Note:

AWREADY stays LOW if buf_valid=1 (TX buffer full). CPU bridge waits — no bytes lost.WSTRB=4'h1 (SB instruction → byte 0 only, not 0xF).

AXI Read Transaction — uart_getc()

AR Address Read

ARADDR = 0x10000008 (STATUS)
or 0x10000004 (RX DATA)
ARVALID = 1

R Read Data

STATUS: RDATA = {30'b0, rx_buf_valid, 1'b0}
RX DATA: RDATA = {23'b0, rx_buf_valid,
rx_data[7:0]}



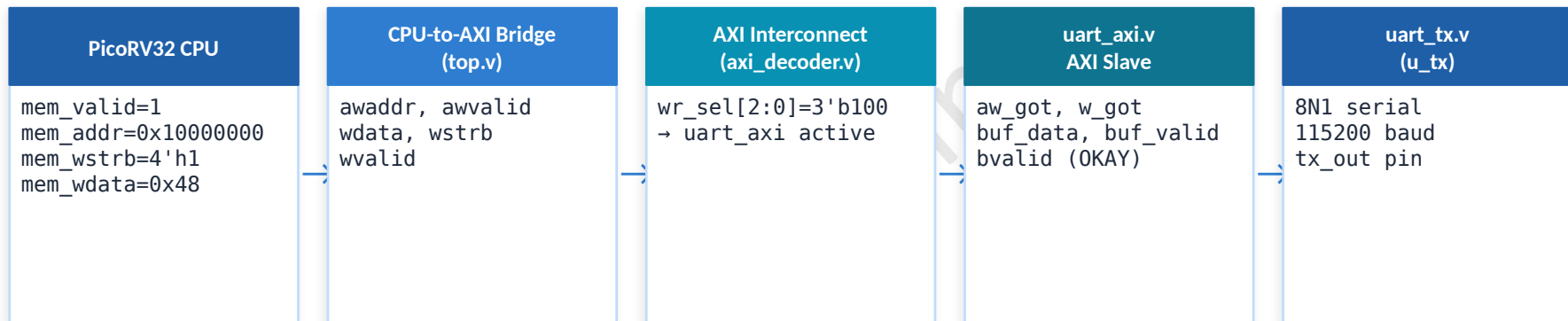
✓ Correct bit interpretation:

bit[1] = rx_buf_valid (NOT bit[31] — that is SiFive FIFO convention, NOT this project)

Firmware polling idiom:

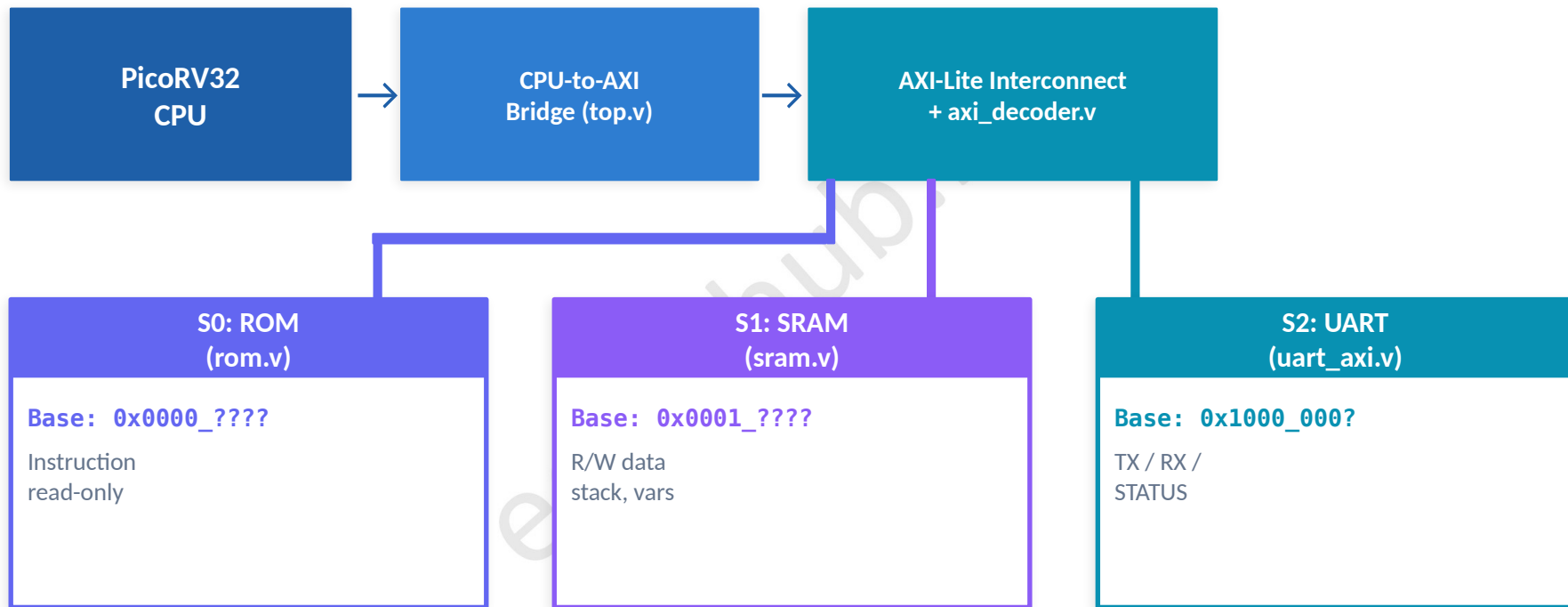
```
while (!(*(volatile uint32_t*)0x10000008 & 0x2)); // wait for rx_buf_valid  
uint8_t c = *(volatile uint32_t*)0x10000004 & 0xFF; // read RX byte
```

Signal Mapping — CPU → AXI → UART



Signal	Direction	Description
mem_wstrb = 4'h1	CPU → Bridge	Byte-write (SB instruction); only byte 0 written
s_axi_awready	uart_axi → Bridge	0 when buf_valid=1 (backpressure)
buf_valid	uart_axi internal	TX buffer occupied; stalls AXI master
tx_out	uart_axi → pin	Renamed from uart_tx to avoid Yosys collision

SoC Subsystem Architecture



uart_axi internals: AXI Write FSM → [buf_data/buf_valid] → uart_tx (u_tx) → tx_out | uart_rx (u_rx) → rx_buf_data/rx_buf_valid → AXI Read FSM

MEMORY MAP - This Project

Address	Slave	Register / Purpose
0x0000_????	ROM	Instruction / data — read-only
0x0001_????	SRAM	R/W data — stack, variables
0x1000_0000	UART	TX: write byte to transmit
0x1000_0004	UART	RX: read received byte
0x1000_0008	UART	STATUS: bit[1]=1 means RX byte ready

DESIGN RULES

1 VALID Stays High Until Handshake

Once VALID is asserted, it must not be deasserted until READY is also high. One outstanding TX in-flight.

2 Backpressure — AWREADY/WREADY

uart_axi sets awready/wready=0 when buf_valid=1. CPU bridge waits — no bytes are ever lost, no FIFO needed.

3 Port Rename: tx_out (not uart_tx)

uart_axi instantiates a submodule named uart_tx. A same-name port causes Yosys to lose the wire driver during synthesis. Renamed to tx_out.

4 ASIC-Clean RTL Constraints

No initial blocks, no inline register initializers in uart_axi.v. All state driven by synchronous reset only. Synthesis-safe.

5 2-FF Synchroniser in uart_rx

Metastability protection on the incoming serial line. Two flip-flops synchronise the async pin before processing.

6 WSTRB = 4'h1 (Byte Write)

SB instruction produces wstrb=4'h1 (byte 0 only), NOT 0xF. uart_axi reads wdata[7:0] only.

End-to-End Walkthrough — `uart_putc('H')`

1 `fw/main.c`

`uart_putc('H') → UART_TX = 0x48`

2 `PicoRV32`

`mem_valid=1, mem_addr=0x10000000, mem_wstrb=4'h1, mem_wdata=0x48`

3 `CPU-to-AXI Bridge`

`awaddr=0x10000000, awvalid=1, wdata=0x48, wstrb=4'h1, wvalid=1`

4 `Interconnect`

`wr_sel=3'b100 → only uart_axi active`

5 `uart_axi.v`

`aw_got=1 → w_got=1 → buf_data=0x48, buf_valid=1`

6 `uart_axi.v`

`bvalid=1 (OKAY) → bridge: cpu_mem_ready=1 → CPU resumes`

7 `uart_tx.v (u_tx)`

`data=0x48, valid=1 → IDLE→START→DATA(×8)→STOP`

8 `Serial wire`

`8N1 frame at 115200 baud → testbench UART monitor decodes 'H'`

9 `tb_top.v logs: [TB] RX byte: H`

Lab — AXI + UART Integration Testbench

\$ make axi_uart

Files Used

rtl/uart_tx.v	– TX serialiser
rtl/uart_rx.v	– RX + 2-FF sync
rtl/uart_axi.v	– AXI slave wrapper
tb/tb_axi_lite_master.v	– AXI master tasks
tb/tb_axi_lite.v	– master + DUT

What Happens

- AXI master writes 0x48 ('H') → 0x10000000
- uart_axi buffers → uart_tx serialises → tx_out
- uart_rx receives (loopback) → rx_buf_valid=1
- Master reads STATUS[1]=1 (RX ready)
- Master reads RX=0x48 → WRITE DATA == READ DATA → PASS

Key GTKWave Signals:

```
s_axi_awaddr / awvalid / awready | s_axi_wdata / wvalid / wready | bvalid | uart_axi.buf_valid |
u_tx.state[1:0] | rx_buf_valid | s_axi_rdata
```

Key Takeaways

uart_axi.v

AXI-Lite slave wrapper for uart_tx.v + uart_rx.v. 250-line ASIC-clean RTL.

TX Backpressure

awready/wready=0 when buf_valid=1 → zero data loss, no FIFO needed.

Port Rename

tx_out (not uart_tx) — renamed to avoid Yosys synthesis scope collision.

Register Map

TX: 0x1000_0000 | RX: 0x1000_0004 | STATUS: 0x1000_0008

RX Status

rx_buf_valid flag in STATUS bit[1]; cleared when RX register is read.

Lab Command

make axi_uart → byte written → UART serialised → loopback → PASS.